

Travel Route Optimizing System:

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_CITIES 10000 // Maximum number of cities
#define MAX_NAME_LENGTH 20 // Maximum length of city names
#define MAX_DISTANCE 1000 // Maximum distance between cities
#define MAX_COST 1000 // Maximum cost for travel

// Arrays for random data generation
const char *CITY_NAMES[] = {"CityA", "CityB", "CityC", "CityD", "CityE", "CityF",
"CityG", "CityH", "CityI", "CityJ"};

// Structure to store travel route details
typedef struct {
    char source[MAX_NAME_LENGTH];
    char destination[MAX_NAME_LENGTH];
    int distance;
    int cost;
} Route;

// Function to generate a random route
void generate_route(FILE *file, int index) {
    char source[MAX_NAME_LENGTH];
    char destination[MAX_NAME_LENGTH];
```

```

int distance = 1 + rand() % MAX_DISTANCE; // Random distance between 1 and 1000
int cost = 1 + rand() % MAX_COST; // Random cost between 1 and 1000

// Generate source and destination cities
int source_index = rand() % (sizeof(CITY_NAMES) / sizeof(CITY_NAMES[0]));
int dest_index;
do {
    dest_index = rand() % (sizeof(CITY_NAMES) / sizeof(CITY_NAMES[0]));
} while (dest_index == source_index); // Ensure source and destination are different

strcpy(source, CITY_NAMES[source_index]);
strcpy(destination, CITY_NAMES[dest_index]);

// Write route details to file
fprintf(file, "%s %s %d %d\n", source, destination, distance, cost);
}

// Function to merge two sorted arrays (used in merge sort)
void merge(Route arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    Route L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

```

```

int i = 0, j = 0, k = l;
while (i < n1 && j < n2) {
    if (L[i].distance <= R[j].distance) {
        arr[k] = L[i];
        i++;
    } else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}

}

// Function to perform merge sort on routes based on distance
void merge_sort(Route arr[], int l, int r) {

```

```

    if (l < r) {
        int m = l + (r - l) / 2;
        merge_sort(arr, l, m);
        merge_sort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

// Function to perform 0/1 Knapsack for selecting routes within a budget
int knapsack(Route routes[], int n, int budget, int selected[]) {
    int **dp = malloc((n + 1) * sizeof(int *));
    for (int i = 0; i <= n; i++) {
        dp[i] = malloc((budget + 1) * sizeof(int));
    }

    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= budget; j++) {
            if (i == 0 || j == 0)
                dp[i][j] = 0;
            else if (routes[i - 1].cost <= j)
                dp[i][j] = (dp[i - 1][j] > dp[i - 1][j - routes[i - 1].cost] + routes[i - 1].distance) ? dp[i - 1][j] : dp[i - 1][j - routes[i - 1].cost] + routes[i - 1].distance;
            else
                dp[i][j] = dp[i - 1][j];
        }
    }

    // Backtrack to find selected routes
    int remaining_budget = budget;

```

```

for (int i = n; i > 0; i--) {
    if (dp[i][remaining_budget] != dp[i - 1][remaining_budget]) {
        selected[i - 1] = 1; // Mark route as selected
        remaining_budget -= routes[i - 1].cost;
    }
}

int result = dp[n][budget];

for (int i = 0; i <= n; i++) {
    free(dp[i]);
}
free(dp);

return result;
}

// Function to perform Coin Change for minimizing the number of routes
int coin_change(Route routes[], int n, int budget, int selected[]) {
    int *dp = malloc((budget + 1) * sizeof(int));
    for (int i = 0; i <= budget; i++) {
        dp[i] = INT_MAX; // Initialize with a large value
    }
    dp[0] = 0;

    for (int i = 1; i <= budget; i++) {
        for (int j = 0; j < n; j++) {
            if (routes[j].cost <= i && dp[i - routes[j].cost] != INT_MAX) {

```

```

        if (dp[i - routes[j].cost] + 1 < dp[i]) {
            dp[i] = dp[i - routes[j].cost] + 1;
        }
    }
}

// Backtrack to find selected routes
int remaining_budget = budget;
for (int i = n - 1; i >= 0; i--) {
    while (remaining_budget >= routes[i].cost && dp[remaining_budget] ==
dp[remaining_budget - routes[i].cost] + 1) {
        selected[i] = 1; // Mark route as selected
        remaining_budget -= routes[i].cost;
    }
}

int result = dp[budget];

free(dp);
return result;
}

// Function to get current time in nanoseconds (for high-resolution timing)
long long get_time_ns() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (long long)ts.tv_sec * 1000000000LL + ts.tv_nsec;
}

```

```

int main() {

    Route *routes = malloc(MAX_CITIES * sizeof(Route)); // Allocate on heap
    int *selected = calloc(MAX_CITIES, sizeof(int)); // Allocate on heap

    int n = 0, budget = 0;
    long long start, end;
    double merge_sort_time, knapsack_time, coin_change_time;

    // Seed the random number generator
    srand(time(NULL));

    // Ask the user for the number of routes
    printf("Enter the number of routes to generate (100 to 10000): ");
    if (scanf("%d", &n) != 1 || n < 100 || n > MAX_CITIES) {
        printf("Invalid input. Please enter a number between 100 and %d.\n", MAX_CITIES);
        return 1;
    }

    // Ask the user for the budget
    printf("Enter the budget for travel: ");
    if (scanf("%d", &budget) != 1 || budget <= 0) {
        printf("Invalid input. Please enter a positive budget.\n");
        return 1;
    }

    // Open the file for writing
    FILE *file = fopen("input.txt", "w");

```

```
if (!file) {
    perror("Error opening file");
    return 1;
}

// Write the budget to file
fprintf(file, "%d\n", budget);

// Generate and write random routes
for (int i = 0; i < n; i++) {
    generate_route(file, i);
}

// Close the file
fclose(file);

// Read input from file
FILE *input_file = fopen("input.txt", "r");
if (!input_file) {
    perror("Error opening input file");
    return 1;
}

// Read budget
fscanf(input_file, "%d", &budget);

// Read route details
n = 0;
```

```
while (fscanf(input_file, "%s %s %d %d", routes[n].source, routes[n].destination,
&routes[n].distance, &routes[n].cost) != EOF) {
    n++;
}
fclose(input_file);
```

```
// Measure Merge Sort execution time using high-resolution timer
```

```
start = get_time_ns();
```

```
merge_sort(routes, 0, n - 1);
```

```
end = get_time_ns();
```

```
merge_sort_time = (end - start) / 1e9; // Convert nanoseconds to seconds
```

```
// Measure Knapsack execution time
```

```
start = get_time_ns();
```

```
int max_distance = knapsack(routes, n, budget, selected);
```

```
end = get_time_ns();
```

```
knapsack_time = (end - start) / 1e9; // Convert nanoseconds to seconds
```

```
// Measure Coin Change execution time
```

```
start = get_time_ns();
```

```
int min_routes = coin_change(routes, n, budget, selected);
```

```
end = get_time_ns();
```

```
coin_change_time = (end - start) / 1e9; // Convert nanoseconds to seconds
```

```
// Write output to file
```

```
FILE *output_file = fopen("output.txt", "w");
```

```
if (!output_file) {
```

```
    perror("Error opening output file");
```

```
    return 1;
```

```
}
```

```
fprintf(output_file, "Selected Routes (Knapsack):\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    if (selected[i]) {
```

```
        fprintf(output_file, "%s -> %s (Distance: %d, Cost: %d)\n", routes[i].source,  
routes[i].destination, routes[i].distance, routes[i].cost);
```

```
    }
```

```
}
```

```
fprintf(output_file, "\nSelected Routes (Coin Change):\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    if (selected[i]) {
```

```
        fprintf(output_file, "%s -> %s (Distance: %d, Cost: %d)\n", routes[i].source,  
routes[i].destination, routes[i].distance, routes[i].cost);
```

```
    }
```

```
}
```

```
fclose(output_file);
```

```
// Write execution times to file
```

```
FILE *time_file = fopen("time_results.txt", "w");
```

```
if (!time_file) {
```

```
    perror("Error opening time results file");
```

```
    return 1;
```

```
}
```

```
fprintf(time_file, "Merge Sort Time: %.9f seconds\n", merge_sort_time);
```

```
fprintf(time_file, "Knapsack Time: %.9f seconds\n", knapsack_time);
```

```
fprintf(time_file, "Coin Change Time: %.9f seconds\n", coin_change_time);  
fclose(time_file);  
  
// Free allocated memory  
free(routes);  
free(selected);  
  
printf("Program executed successfully. Check output.txt and time_results.txt for results.\n");  
  
return 0;  
}
```